C PROGRAMMING LAB REPORT

HDOC Day 11

Experiments

1. Roots of a Quadratic Equation
2. Menu-Driven Income Tax Calculator

This report contains detailed aims, objectives, theory, formulas, algorithms, C programs, sample inputs and outputs, results, and precautions.

EXPERIMENT 1: ROOTS OF A QUADRATIC EQUATION

Aim

To write a C program to find the roots of a quadratic equation a*x^2 + b*x + c = 0 and determine whether the roots are real and distinct, real and equal, or complex.

Objectives

1. To understand variables and floating-point data types.
2. To accept coefficients using scanf().
3. To calculate the discriminant.
4. To use if-else statements for different cases.
5. To use sqrt() from the math library.
6. To display the calculated roots.

Theory

A quadratic equation has the form a*x^2 + b*x + c = 0, where a != 0. The nature of the roots is determined by its discriminant D.

```
D = b^2 - 4*a*c
```

Condition	Nature of roots
D > 0	Two real and distinct roots
D == 0	Two real and equal roots
D < 0	Two complex roots

Formulas

```
If D > 0:
x1 = (-b + sqrt(D)) / (2*a)
x2 = (-b - sqrt(D)) / (2*a)

If D == 0:
x1 = x2 = -b / (2*a)

If D < 0:
Real Part = -b / (2*a)
Imaginary Part = sqrt(-D) / (2*a)
```

Detailed Algorithm

1. Start the program.
2. Include stdio.h for input and output operations.
3. Include math.h because sqrt() is required for calculating square roots.
4. Declare variables a, b, c and D to store the coefficients and discriminant.
5. Declare root1 and root2 for real roots.
6. Declare realPart and imagPart for complex roots.
7. Ask the user to enter a, b and c.
8. Read the three coefficients using scanf().
9. Check whether a == 0.
10. If a == 0, display that the equation is not quadratic, because the coefficient of x^2 must be non-zero, and terminate the calculation.

11. If a != 0, calculate D using D = b^2 - 4*a*c.
12. Check whether D > 0.
13. If D > 0, conclude that the equation has two real and distinct roots.
14. Calculate root1 using root1 = (-b + sqrt(D)) / (2*a).
15. Calculate root2 using root2 = (-b - sqrt(D)) / (2*a).
16. Display root1 and root2.
17. If D is not greater than zero, check whether D == 0.
18. If D == 0, conclude that the equation has two real and equal roots.
19. Calculate the common root using root1 = -b / (2*a).
20. Display the common value as both roots.
21. If D is neither greater than zero nor equal to zero, then D < 0.
22. For D < 0, conclude that the roots are complex.
23. Calculate the real part using realPart = -b / (2*a).
24. Calculate the imaginary part using imagPart = sqrt(-D) / (2*a).
25. Display the roots as realPart + imagPart*i and realPart - imagPart*i.
26. Return from main() and terminate the program.
27. Stop.

C Program

```
#include <stdio.h>
#include <math.h>

int main()
{
    float a, b, c, D;
    float root1, root2;
    float realPart, imagPart;

    printf("Enter a, b and c: ");
    scanf("%f %f %f", &a, &b, &c);

    if (a == 0)
    {
        printf("It is not a quadratic equation.\n");
    }
    else
    {
        D = b * b - 4 * a * c;

        if (D > 0)
        {
            root1 = (-b + sqrt(D)) / (2 * a);
            root2 = (-b - sqrt(D)) / (2 * a);

            printf("Two real and distinct roots:\n");
            printf("Root 1 = %.2f\n", root1);
            printf("Root 2 = %.2f\n", root2);
        }
        else if (D == 0)
        {
            root1 = -b / (2 * a);

            printf("Two real and equal roots:\n");
            printf("Root 1 = Root 2 = %.2f\n", root1);
        }
        else
        {
            realPart = -b / (2 * a);
            imagPart = sqrt(-D) / (2 * a);
```

```
            printf("Two complex roots:\n");
            printf("Root 1 = %.2f + %.2fi\n", realPart, imagPart);
            printf("Root 2 = %.2f - %.2fi\n", realPart, imagPart);
        }
    }

    return 0;
}
```

Sample Input

```
Enter a, b and c: 1 -5 6
```

Sample Calculation

```
D = (-5)^2 - 4*1*6
D = 25 - 24
D = 1

Since D > 0, roots are real and distinct.
Root 1 = 3.00
Root 2 = 2.00
```

Sample Output

```
Two real and distinct roots:
Root 1 = 3.00
Root 2 = 2.00
```

Test Cases

a	b	c	D	Expected result
1	-5	6	1	Roots = 3.00, 2.00
1	2	1	0	Both roots = -1.00
1	2	5	-16	Complex roots = -1.00 +/- 2.00i
0	2	1	-	Not a quadratic equation

Result

The program successfully calculates the roots of a quadratic equation and correctly identifies the nature of the roots.

EXPERIMENT 2: MENU-DRIVEN INCOME TAX CALCULATOR

Aim

To write a menu-driven C program to calculate income tax under the given old and new tax regime rules, including deductions, rebate, and Health and Education Cess.

Objectives

1. To understand menu-driven programming.
2. To use loops and conditional statements.
3. To accept taxpayer details.
4. To calculate taxable income after deductions.
5. To apply age-based old-regime slabs.
6. To calculate tax using new-regime slabs.
7. To apply rebate and cess.

Old Regime Deduction Limits

Deduction	Maximum allowed
Standard Deduction	Rs. 50,000
Savings Deduction	Rs. 1,50,000
Pension Deduction	Rs. 50,000
Medical Deduction	Rs. 25,000
Home Loan Interest	Rs. 2,00,000
Savings Account Interest	Rs. 10,000
Senior Citizen Interest	Rs. 50,000

Taxable Income

```
Total Deduction = Sum of all allowed deductions
Taxable Income = Gross Income - Total Deduction
If Taxable Income < 0:
    Taxable Income = 0
```

Old Regime Tax Slabs

Age category	Income range	Rate
Below 60	Up to Rs. 2,50,000	Nil
Below 60	Rs. 2,50,001 - 5,00,000	5%
Below 60	Rs. 5,00,001 - 10,00,000	20%
Below 60	Above Rs. 10,00,000	30%
60 to below 80	Up to Rs. 3,00,000	Nil
60 to below 80	Rs. 3,00,001 - 5,00,000	5%
60 to below 80	Rs. 5,00,001 - 10,00,000	20%
60 to below 80	Above Rs. 10,00,000	30%
80 or above	Up to Rs. 5,00,000	Nil

Age category	Income range	Rate
80 or above	Rs. 5,00,001 - 10,00,000	20%
80 or above	Above Rs. 10,00,000	30%

Old-regime rebate: if taxable income <= Rs. 5,00,000, rebate = the smaller of Rs. 12,500 and calculated tax.

New Regime Tax Slabs

Income range	Rate
Up to Rs. 4,00,000	Nil
Rs. 4,00,001 - 8,00,000	5%
Rs. 8,00,001 - 12,00,000	10%
Rs. 12,00,001 - 16,00,000	15%
Rs. 16,00,001 - 20,00,000	20%
Rs. 20,00,001 - 24,00,000	25%
Above Rs. 24,00,000	30%

New-regime rebate: if taxable income <= Rs. 12,00,000, rebate = the smaller of Rs. 60,000 and calculated tax.

Final Calculation

```
Tax After Rebate = Tax Before Rebate - Rebate
Cess = 0.04 * Tax After Rebate
Final Tax Payable = Tax After Rebate + Cess
```

Detailed Algorithm

1. Start the program.
2. Include the required header files for input/output and mathematical functions.
3. Declare a character array to store the taxpayer's name.
4. Declare variables for age, gross income, total deduction, taxable income, tax, rebate, cess, and final tax.
5. Declare separate variables for standard deduction, savings deduction, pension deduction, medical deduction, home loan interest, savings account interest, and senior citizen interest.
6. Display a prompt asking the user to enter the taxpayer's name and read it.
7. Ask the user to enter age and store the value.
8. Ask the user to enter gross annual income and store the value.
9. Display the menu with three choices: 1 for Old Tax Regime, 2 for New Tax Regime, and 3 for Exit.
10. Read the user's menu choice.
11. If the choice is 1, select the old tax regime.
12. Ask the user to enter every deduction applicable under the old regime.
13. Compare standard deduction with Rs. 50,000. If it is greater, use Rs. 50,000 as the allowed value.
14. Compare savings deduction with Rs. 1,50,000 and use only the permitted amount.
15. Compare pension deduction with Rs. 50,000 and use only the permitted amount.
16. Compare medical deduction with Rs. 25,000 and use only the permitted amount.
17. Compare home loan interest with Rs. 2,00,000 and use only the permitted amount.

18. Compare savings account interest with Rs. 10,000 and use only the permitted amount.
19. Compare senior citizen interest with Rs. 50,000 and use only the permitted amount.
20. Add all allowed deductions to obtain total deduction.
21. Calculate taxable income using Taxable Income = Gross Income - Total Deduction.
22. If taxable income is negative, set it to zero.
23. Determine whether the taxpayer is below 60, between 60 and below 80, or 80 and above.
24. Apply the appropriate old-regime slabs according to the taxpayer's age category.
25. Calculate tax progressively, applying each percentage only to the income portion belonging to that slab.
26. Store the calculated amount as tax before rebate.
27. Check whether taxable income <= Rs. 5,00,000.
28. If the condition is true, calculate rebate as the smaller of Rs. 12,500 and tax before rebate; otherwise set rebate to zero.
29. If the choice is 2, select the new tax regime.
30. Under the given assignment rules, set taxable income equal to gross income for the new regime.
31. Apply the new-regime slabs beginning with the Nil slab up to Rs. 4,00,000.
32. Apply 5% to the applicable portion from Rs. 4,00,001 to Rs. 8,00,000.
33. Apply 10% to the applicable portion from Rs. 8,00,001 to Rs. 12,00,000.
34. Apply 15% to the applicable portion from Rs. 12,00,001 to Rs. 16,00,000.
35. Apply 20% to the applicable portion from Rs. 16,00,001 to Rs. 20,00,000.
36. Apply 25% to the applicable portion from Rs. 20,00,001 to Rs. 24,00,000.
37. Apply 30% to the portion above Rs. 24,00,000.
38. Add the tax from all applicable slabs to obtain tax before rebate.
39. Check whether taxable income <= Rs. 12,00,000.
40. If the condition is true, calculate rebate as the smaller of Rs. 60,000 and tax before rebate; otherwise set rebate to zero.
41. Subtract rebate from tax before rebate to obtain tax after rebate.
42. Calculate cess using Cess = 0.04 * Tax After Rebate.
43. Add cess to tax after rebate to obtain final tax payable.
44. Display taxpayer name, taxable income, tax before rebate, rebate, cess, and final tax payable.
45. After displaying the result, return to the menu so another calculation can be performed.
46. If the choice is 3, display an exit message and terminate the program.
47. If the choice is not 1, 2, or 3, display an invalid-choice message and return to the menu.
48. Continue the menu-driven process until the user selects Exit.
49. Stop the program.

C Program

```
#include <stdio.h>
#include <math.h>

float oldTax(float income, int age)
{
    float tax = 0;
```

```
    if (age >= 80)
    {
        if (income > 1000000)
            tax += (income - 1000000) * 0.30;

        if (income > 500000)
            tax += (income > 1000000 ? 500000 : income - 500000) * 0.20;
    }
    else if (age >= 60)
    {
        if (income > 1000000)
            tax += (income - 1000000) * 0.30;

        if (income > 500000)
            tax += (income > 1000000 ? 500000 : income - 500000) * 0.20;

        if (income > 300000)
            tax += (income > 500000 ? 200000 : income - 300000) * 0.05;
    }
    else
    {
        if (income > 1000000)
            tax += (income - 1000000) * 0.30;

        if (income > 500000)
            tax += (income > 1000000 ? 500000 : income - 500000) * 0.20;

        if (income > 250000)
            tax += (income > 500000 ? 250000 : income - 250000) * 0.05;
    }

    return tax;
}

float newTax(float income)
{
    float tax = 0;

    if (income > 2400000)
        tax += (income - 2400000) * 0.30;
    if (income > 2000000)
        tax += (income > 2400000 ? 400000 : income - 2000000) * 0.25;
    if (income > 1600000)
        tax += (income > 2000000 ? 400000 : income - 1600000) * 0.20;
    if (income > 1200000)
        tax += (income > 1600000 ? 400000 : income - 1200000) * 0.15;
    if (income > 800000)
        tax += (income > 1200000 ? 400000 : income - 800000) * 0.10;
    if (income > 400000)
        tax += (income > 800000 ? 400000 : income - 400000) * 0.05;

    return tax;
}

int main()
{
    char name[50];
    int age, choice;
    float income, totalDeduction, taxableIncome;
    float standard, savings, pension, medical;
    float homeLoan, savingsInterest, seniorInterest;
    float tax, rebate, cess, finalTax;

    while (1)
    {
        printf("\nEnter taxpayer name: ");
        scanf(" %49[^\n]", name);

        printf("Enter age: ");
        scanf("%d", &age);

        printf("Enter gross income: ");
        scanf("%f", &income);

        printf("\n1. Old Tax Regime\n");
        printf("2. New Tax Regime\n");
```

```
    printf("3. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        printf("Standard Deduction: ");
        scanf("%f", &standard);
        printf("Savings Deduction: ");
        scanf("%f", &savings);
        printf("Pension Deduction: ");
        scanf("%f", &pension);
        printf("Medical Deduction: ");
        scanf("%f", &medical);
        printf("Home Loan Interest: ");
        scanf("%f", &homeLoan);
        printf("Savings Account Interest: ");
        scanf("%f", &savingsInterest);
        printf("Senior Citizen Interest: ");
        scanf("%f", &seniorInterest);

        if (standard > 50000) standard = 50000;
        if (savings > 150000) savings = 150000;
        if (pension > 50000) pension = 50000;
        if (medical > 25000) medical = 25000;
        if (homeLoan > 200000) homeLoan = 200000;
        if (savingsInterest > 10000) savingsInterest = 10000;
        if (seniorInterest > 50000) seniorInterest = 50000;

        totalDeduction = standard + savings + pension + medical
                       + homeLoan + savingsInterest + seniorInterest;

        taxableIncome = income - totalDeduction;

        if (taxableIncome < 0)
            taxableIncome = 0;

        tax = oldTax(taxableIncome, age);

        if (taxableIncome <= 500000)
            rebate = fmin(12500, tax);
        else
            rebate = 0;
    }
    else if (choice == 2)
    {
        taxableIncome = income;
        tax = newTax(taxableIncome);

        if (taxableIncome <= 1200000)
            rebate = fmin(60000, tax);
        else
            rebate = 0;
    }
    else if (choice == 3)
    {
        printf("Exiting program...\n");
        break;
    }
    else
    {
        printf("Invalid choice. Try again.\n");
        continue;
    }

    tax = tax - rebate;
    cess = 0.04 * tax;
    finalTax = tax + cess;

    printf("\nTaxpayer Name: %s\n", name);
    printf("Taxable Income: Rs. %.2f\n", taxableIncome);
    printf("Tax Before Rebate: Rs. %.2f\n", tax + rebate);
    printf("Rebate: Rs. %.2f\n", rebate);
    printf("Cess: Rs. %.2f\n", cess);
    printf("Final Tax Payable: Rs. %.2f\n", finalTax);
}
```

```
    return 0;
}
```

Sample Input - Old Tax Regime

```
Name: Anushka
Age: 35
Gross Income: 900000
Choice: 1
Standard Deduction: 50000
Savings Deduction: 100000
Pension Deduction: 20000
Medical Deduction: 15000
Home Loan Interest: 50000
Savings Account Interest: 5000
Senior Citizen Interest: 0
```

Expected Calculation

```
Total Deduction = Rs. 2,40,000
Taxable Income = Rs. 6,60,000
Tax Before Rebate = Rs. 47,000
Rebate = Rs. 0
Cess = Rs. 1,880
Final Tax Payable = Rs. 48,880
```

Sample Input - New Tax Regime

```
Name: Anushka
Age: 30
Gross Income: 1100000
Choice: 2
```

Expected Calculation

```
Taxable Income = Rs. 11,00,000
Tax Before Rebate = Rs. 50,000
Rebate = Rs. 50,000
Tax After Rebate = Rs. 0
Cess = Rs. 0
Final Tax Payable = Rs. 0
```

Result

The menu-driven C program successfully calculates taxable income and tax payable under the given old and new regime rules, including deductions, rebate, and Health and Education Cess.

Precautions

1. Enter valid numerical values for age, income, and deductions.
2. Apply deduction limits correctly.
3. Use the correct age category for the old regime.
4. Apply tax slabs progressively and in the correct order.
5. Apply rebate only when the specified condition is satisfied.
6. Calculate cess after rebate.
7. Since fmin() is used, compile this program with the math library, for example: gcc expp5.c -o expp5 -lm.

Overall Conclusion

Both experiments demonstrate important C programming concepts including variables, input/output, arithmetic operations, conditional statements, functions, loops, and mathematical library functions. The first experiment determines quadratic roots using the discriminant, while the second demonstrates a menu-driven application involving progressive calculations and decision making.